# Network Intrusion Detection & Traffic Forensics Guide

Complete Technical Breakdown of Project 2 Attack Vectors, Suricata Signatures, Zeek Telemetry & OWASP/ MITRE Mapping

**STACK: SURICATA NIDS • ZEEK • WIRESHARK • LINUX VIRTUAL BRIDGE • PCAP ANALYSIS**

## 1. Architectural Context: OWASP Top 10 vs. Network Attacks

Before investigating packets, it is critical to clarify a common industry misconception regarding security frameworks—especially for technical interviews:

**Crucial Framework Distinction: OWASP Top 10 vs. MITRE ATT&CK**

There is **no official "OWASP Top 10 for Network Attacks."** The **OWASP Top 10** (2021 and the updated 2025 release) is strictly a **Web Application Security (OSI Layer 7)** framework. For low-level network intrusions, C2 beaconing, port scanning, and protocol tunneling (OSI Layers 3, 4, and 7 network transport), enterprise SOC teams use the **MITRE ATT&CK Framework**.

However, many network attacks analyzed in Project 2 directly exploit or trigger **OWASP Top 10 Application & API categories** when crossing the wire. Both mappings are detailed below so you can answer any interview question accurately.

| Project 2 Attack Vector | OSI Layer | MITRE ATT&CK ID | Related OWASP Top 10 (2025 / API) Mapping |
|---|---|---|---|
| **1. Raw SYN Port Scans** | Layer 4 (TCP) | T1046 (Network Service Discovery) | **A02:2025 Security Misconfiguration** (Exposed ports/services) |
| **2. Base64 HTTP User-Agents** | Layer 7 (HTTP) | T1071.001 (Web Protocols)<br>T1132.001 (Standard Encoding) | **A05:2025 Injection** & **A09:2025 Logging/ Alerting Failures** (Unsanitized headers / C2 evasion) |
| **3. SSH Egress Tunnels** | Layer 4/7 (SSH) | T1572 (Protocol Tunneling)<br>T1021.004 (SSH) | **A01:2025 Broken Access Control** & **A02:2025 Misconfiguration** (Unrestricted outbound firewall egress) |
| **4. C2 Beaconing (Cobalt Strike / Sliver)** | Layer 4/7 (HTTPS/TCP) | T1071 (App Layer Protocol)<br>T1573.002 (Asymmetric Crypto) | **A08:2025 Software/Data Integrity Failures** & **A09:2025 Security Logging Failures** |
| **5. Fake / Rogue TLS Certificates** | Layer 6/7 (TLS/ SSL) | T1587.003 (Digital Certificates)<br>T1573 (Encrypted Channel) | **A04:2025 Cryptographic Failures** & **A07:2025 Authentication Failures** (Self-signed/untrusted certs) |
| **6. DNS Exfiltration Tunnels** | Layer 7 (DNS) | T1048.003 (Exfiltration Over Unencrypted Non-C2 Protocol) | **A02:2025 Security Misconfiguration** (Unmonitored external DNS resolution) |

## 2. Deep-Dive Breakdown of Project 2 Attack Vectors

### Attack 1: Raw SYN Port Scanning (Stealth / Half-Open Recon)

**MITRE: T1046** **OWASP: A02:2025 Security Misconfiguration**

**How It Works Simply:** Normally, establishing a TCP connection requires a 3-way handshake: the client sends a `SYN` packet, the server replies with `SYN-ACK` (if the port is open), and the client completes it with an `ACK`. In a

**Raw SYN Scan** (like `nmap -sS`), the attacker rapidly blasts thousands of `SYN` packets at different ports on a target server. The moment a port replies with `SYN-ACK`, the attacker knows the port is open and immediately sends a `RST` (Reset) packet to tear down the connection before completing the handshake.

**Why Attackers Use It:** Because the full TCP 3-way handshake is never completed, older application-layer logs do not record the connection, making it a fast way to discover misconfigured open services (OWASP A02).

**How You Detect & Investigate It:**

- **Suricata Signature:** Write a rule matching TCP packets with only the SYN flag set (`flags:S;`) using a `detection_filter` that fires when a single source IP sends more than 20 SYN packets within 5 seconds.
- **Zeek (`conn.log`):** Look for hundreds of entries from the same source IP showing connection state `S0` (SYN sent, no reply seen) or `REJ` (connection rejected), with `0` payload bytes.
- **Wireshark Forensics:** Filter with `tcp.flags.syn == 1 and tcp.flags.ack == 0` to observe sequential or randomized destination port knocking.

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"CUSTOM NIDS - Possible Raw SYN Port Scan";
flags:S,12; flow:stateless; detection_filter:track by_src, count 25, seconds 5; sid:1000001;
rev:1;)
```

## Attack 2: Obfuscated Base64 HTTP User-Agent

**MITRE: T1071.001 / T1132.001** **OWASP: A05:2025 Injection / A09:2025 Logging Failures**

**How It Works Simply:** Every time a web browser visits a website, it sends an HTTP `User-Agent` header identifying itself (e.g., `Mozilla/5.0...`). When malware infects an internal host, it disguises its communication as normal HTTP web traffic. Instead of sending plaintext commands or binary data that would break HTTP formatting rules, it encodes stolen data or command requests into **Base64** (using only safe characters `A-Z, a-z, 0-9, +, /, =`) and places them inside the `User-Agent` header.

**Why Attackers Use It & OWASP Relevance:** Corporate firewalls almost always allow HTTP traffic out, and rarely inspect the contents of the `User-Agent` string. Furthermore, in web attacks (**OWASP A05 Injection** & **A09 Logging Failures**), attackers inject payloads into `User-Agent` headers knowing the backend server will log or process the header without sanitizing it.

**How You Detect & Investigate It:**

- **Suricata Signature:** Inspect the `http.user_agent` buffer using PCRE (Perl Compatible Regular Expressions) to flag long, continuous Base64 character blocks ending in padding (`=` or `==`).
- **Zeek (`http.log`):** Inspect the `user_agent` column for anomalous strings that lack spaces or standard browser tokens, then decode them in your terminal via `echo "string" | base64 -d`.
- **Wireshark Forensics:** Right-click the suspicious HTTP packet → *Follow* → *HTTP Stream* to view the full raw headers and server response.

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"CUSTOM NIDS - Suspicious Base64 Encoded
User-Agent"; flow:established,to_server; http.user_agent; pcre:"/^[A-Za-z0-9+\/]{16,}={0,2}$/";
sid:1000002; rev:1;)
```

## Attack 3: Unauthorized SSH Egress Tunnels (Reverse Port Forwarding)

**MITRE: T1572 / T1021.004** **OWASP: A01:2025 Broken Access Control**

**How It Works Simply:** Firewalls block external attackers from connecting *inbound* to internal servers, but often permit internal machines to initiate *outbound* (egress) connections. Once an attacker compromises an internal machine, they initiate an outbound SSH connection (`ssh -R`) from the victim machine to the attacker's external server. Once this encrypted tunnel is established, the attacker uses **Remote Port Forwarding** to send commands and additional malware *backward* down the existing outbound pipe, bypassing inbound firewall blocks entirely.

**Why Attackers Use It:** SSH encryption is rock-solid; the firewall cannot inspect what files or commands are traveling inside the tunnel. It exploits overly permissive outbound network access controls (**OWASP A01** / **A02**).

**How You Detect & Investigate It:**

- **Suricata Signature:** Detect the cleartext SSH protocol banner (`SSH-2.0-`) moving from `$HOME_NET` to `$EXTERNAL_NET` on port 22 or non-standard ports (like port 443 or 8080).
- **Zeek (`ssh.log` & `conn.log`):** Look for outbound SSH connections to unknown external IPs with long durations and high bidirectional byte counts (`orig_bytes` and `resp_bytes`).

```
alert ssh $HOME_NET any -> $EXTERNAL_NET any (msg:"CUSTOM NIDS - Unauthorized Outbound SSH
Egress Tunnel"; flow:established,to_server; sid:1000003; rev:1;)
```

## Attack 4: Command & Control (C2) Beaconing (Cobalt Strike & Sliver)

**MITRE: T1071 / T1095** **OWASP: A08:2025 Integrity Failures / A09:2025 Logging**

**How It Works Simply:** After initial compromise, malware implants generated by frameworks like **Cobalt Strike** (commercial red-team tool heavily abused by ransomware groups) or **Sliver** (modern Go-based C2 framework) do not maintain a noisy constant connection. Instead, they "sleep" and wake up at fixed intervals (e.g., every 60 seconds with a 10% random jitter) to send a short heartbeat ("beacon") to the C2 server asking: *"Do you have any commands for me?"*

**How You Detect & Investigate It:**

- **Suricata Signature:** Match known Cobalt Strike default HTTP URI patterns (such as `/submit.php?id=` or anomalous cookie headers) or JA3/JA3S TLS client fingerprints.
- **Zeek (`conn.log`):** Filter connections between an internal IP and external IP and examine the timestamps (`ts`). Regular, rhythmic time intervals with identical packet sizes prove automated C2 beaconing rather than human web browsing.
- **Wireshark Payload Extraction:** If the staging payload is downloaded over unencrypted HTTP, use *File → Export Objects → HTTP* in Wireshark to carve out the exact malicious executable/shellcode binary and hash it (`sha256sum`).

## Attack 5: Malicious / Self-Signed TLS Certificates in C2 Traffic

**MITRE: T1587.003 / T1573.002** **OWASP: A04:2025 Cryptographic Failures**

**How It Works Simply:** Attackers use HTTPS (TLS) so defenders cannot read their C2 commands in Wireshark. However, before encryption starts, the **TLS Handshake** occurs in plaintext. During this handshake, the attacker's server must present an X.509 digital certificate. Because attackers spin up disposable servers rapidly, they often use self-signed certificates, expired certificates, blank Subject fields, or the hardcoded default certificate bundled inside Cobalt Strike (`Serial Number: 146473198`).

**OWASP Relevance:** Directly maps to **OWASP A04:2025 Cryptographic Failures** and **A07 Authentication Failures**, as trust validation is bypassed using unverified cryptographic identities.

**How You Detect & Investigate It:**

- **Zeek (`ssl.log`):** Inspect the `validation_status` column for `self signed certificate in certificate chain`, check the `subject` and `issuer` fields for anomalies (empty values like `CN=, OU=, O=`), and isolate the certificate serial number.
- **Wireshark Forensics:** Filter by `tls.handshake.type == 11` (Certificate packet), expand *Transport Layer Security → Handshake Protocol: Certificate*, and right-click to export the raw `.der` certificate for forensic evidence.

```
alert tls $EXTERNAL_NET any -> $HOME_NET any (msg:"CUSTOM NIDS - Default Cobalt Strike TLS Certificate Detected"; flow:established,to_client; tls.cert_serial; content:"08:bb:00:ee"; sid:1000004; rev:1;)
```

### Attack 6: DNS Exfiltration & Tunneling

**MITRE: T1048.003 / T1071.004** **OWASP: A02:2025 Security Misconfiguration**

**How It Works Simply:** Even the strictest networks allow DNS (UDP Port 53) so computers can resolve domain names into IP addresses. Attackers exploit this by registering a malicious domain (e.g., `evil-c2.com`) and configuring their own server as its authoritative DNS server. To steal sensitive files from the victim machine, the malware chops the stolen file into encoded chunks and prepends them as subdomains in fake DNS lookups:

```
Query 1: 4a6f686e536d697468506173737 6f7264.evil-c2.com
Query 2: 313233343536536373265744b65793939.evil-c2.com
```

When the local DNS resolver forwards this request to `evil-c2.com`'s name server, the attacker's server reads the subdomain queries and reassembles the stolen file without a single direct connection ever occurring between the victim and the attacker.

**How You Detect & Investigate It:**

- **Suricata Signature:** Alert on abnormally long DNS queries (>50 bytes) or high volumes of `TXT` / `NULL` record lookups to the same root domain.
- **Zeek (`dns.log`):** Extract the `query` column, filter for unusually long high-entropy subdomain labels, strip the root domain, and decode the hex/Base64 strings to reconstruct what data the attacker stole.
- **Wireshark Forensics:** Filter with `dns.qry.name.len > 50` to immediately isolate exfiltration packets from normal web lookups.

```
alert dns $HOME_NET any -> any 53 (msg:"CUSTOM NIDS - Potential DNS Exfiltration Long Subdomain Query"; dns.query; content:"."; pcre:"/^[a-zA-Z0-9]{35,}\./"; sid:1000005; rev:1;)
```

## 3. Complete Forensic Workflow Cheat Sheet (Step-by-Step)

When you sit down to execute Project 2 and document your GitHub write-up, follow this exact 4-stage pipeline:

| Stage | Tool & Command | What You Record for Your Portfolio Report |
|---|---|---|
| **1. Traffic Replay** | `sudo tcpreplay -i virbr1 sample.pcap` | Screenshot of traffic being injected across the isolated Linux bridge while Suricata listens live. |
| **2. NIDS Alerting** | `tail -f /var/log/suricata/fast.log` | Screenshot of your custom `.rules` firing with the exact `sid`, attacker IP, and victim IP. |

| Stage | Tool & Command | What You Record for Your Portfolio Report |
| --- | --- | --- |
| **3. Zeek Hunting** | `zeek -r sample.pcap` `zeek-cut id.orig_h query < dns.log` | Parsed extracts from `conn.log` (beaconing intervals), `dns.log` (exfiltrated strings), and `ssl.log` (fake certs). |
| **4. Deep Forensics** | `wireshark sample.pcap` | TCP stream reconstruction, exported rogue TLS certificate details, and SHA-256 hash of extracted binary payloads. |